



AROR UNIVERSITY
OF ART, ARCHITECTURE,
DESIGN & HERITAGE,
SUKKUR, SINDH

Architectural Design

Prof. Dr. Fahim Akhtar Rajput





Objectives

- Understand why the architectural design of software is important;
- Understand the decisions that have to be made about the software architecture during the architectural design process;
- Have been introduced to the idea of Architectural patterns, well-tried ways of organizing software architectures that can be reused in system designs;
- Understand how Application-Specific Architectural patterns may be used in transaction processing and language processing systems.



Software architecture

- The design process for identifying the subsystems making up a system and the framework for sub-system control and communication is architectural design.
- The output of this design process is a description of the software architecture.

Why is Architecture Important?!

- **Representations of software architecture are an enabler** for communication between all parties (stakeholders) interested in the development of a computer-based system.!
- **The architecture highlights early design decisions** that will have a profound impact on all software engineering work that follows and, as important, on the ultimate success of the system as an operational entity.!
- **Architecture “constitutes a relatively small, intellectually graspable mode** of how the system is structured and how its components work together” [BAS03].!

Architectural design

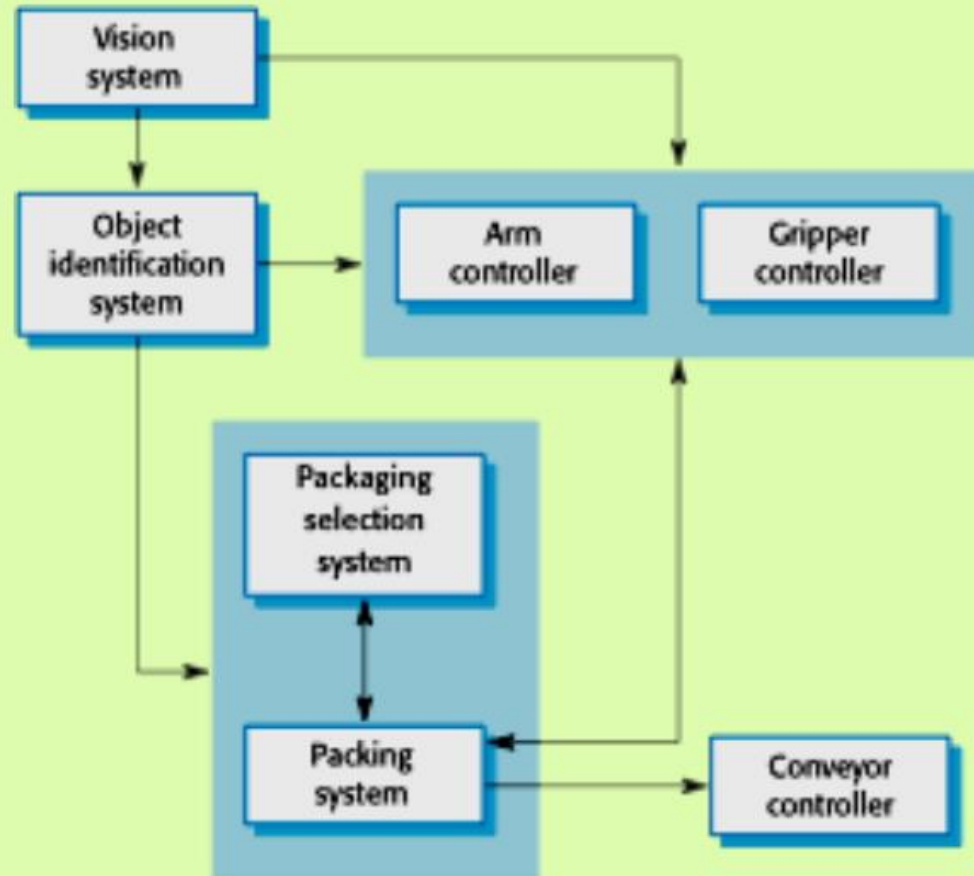
- Architectural design is concerned with understanding how a software system should be organized and designing the overall structure of that system.
- Architectural design is the critical link between design and requirements engineering, as it identifies the main structural components in a system and the relationships between them.
- The output of the architectural design process is an architectural model that describes how the system is organized as a set of communicating components.



Agility and architecture

- It is generally accepted that an early stage of agile processes is to design an overall systems architecture.
- Refactoring the system architecture is usually expensive because it affects so many components in the system

The architecture of a packing robot control system





Architectural abstraction

- **Architecture in the small** is concerned with the architecture of individual programs. At this level, we are concerned with the way that an individual program is decomposed into components.
- **Architecture in the large** is concerned with the architecture of complex enterprise systems that include other systems, programs, and program components. These enterprise systems are distributed over different computers, which may be owned and managed by different companies.



Advantages of explicit architecture

- **Stakeholder communication**
 - Architecture may be used as a focus of discussion by system stakeholders.
- **System analysis**
 - Means that analysis of whether the system can meet its non-functional requirements is possible.
- **Large-scale reuse**
 - The architecture may be reusable across a range of systems
 - Product-line architectures may be developed.



Use of architectural models

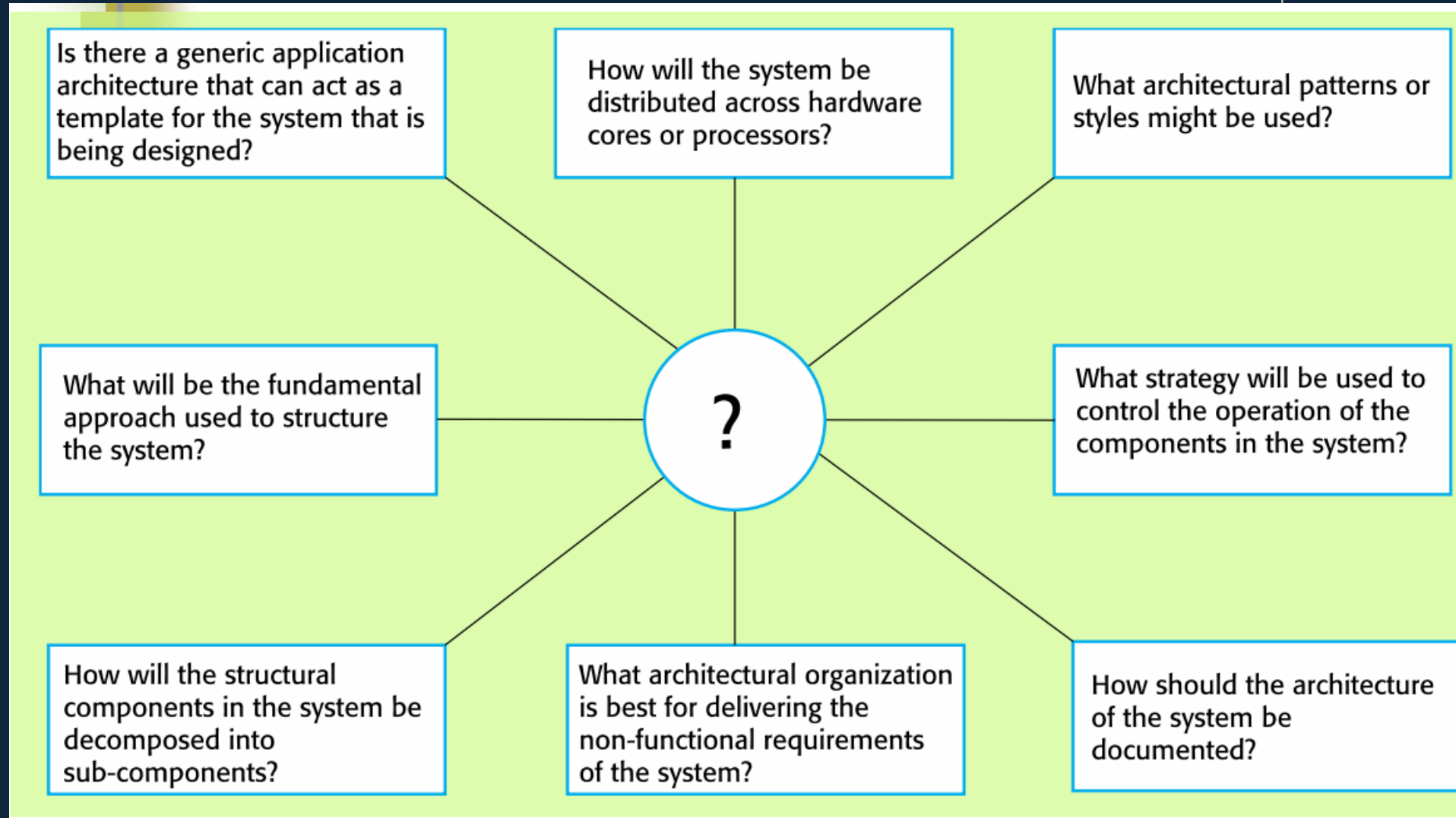
- **As a way of facilitating discussion about the system design**
 - A high-level architectural view of a system is useful for communication with system stakeholders and project planning because it is not cluttered with detail. Stakeholders can relate to it and understand an abstract view of the system. They can then discuss the system as a whole without being confused by detail.
- **As a way of documenting an architecture that has been designed**
 - The aim here is to produce a complete system model that shows the different components in a system, their interfaces and their connections.



Architectural design decisions

- Architectural design is a creative process so the process differs depending on the type of system being developed.
- However, a number of common decisions span all design processes and these decisions affect the non-functional characteristics of the system.

Architectural design decisions



Architectural design decisions



- Is there a generic application architecture that can be used?
- How will the system be distributed?
- What architectural styles are appropriate?
- What approach will be used to structure the system?
- How will the system be decomposed into modules?
- What control strategy should be used?
- How will the architectural design be evaluated?
- How should the architecture be documented?



Architecture reuse

- Systems in the same domain often have similar architectures that reflect domain concepts.
- Application product lines are built around a core architecture with variants that satisfy particular customer requirements.
- The architecture of a system may be designed around one of more architectural patterns or ‘styles’.
 - These capture the essence of an architecture and can be instantiated in different ways.



Architecture and system characteristics

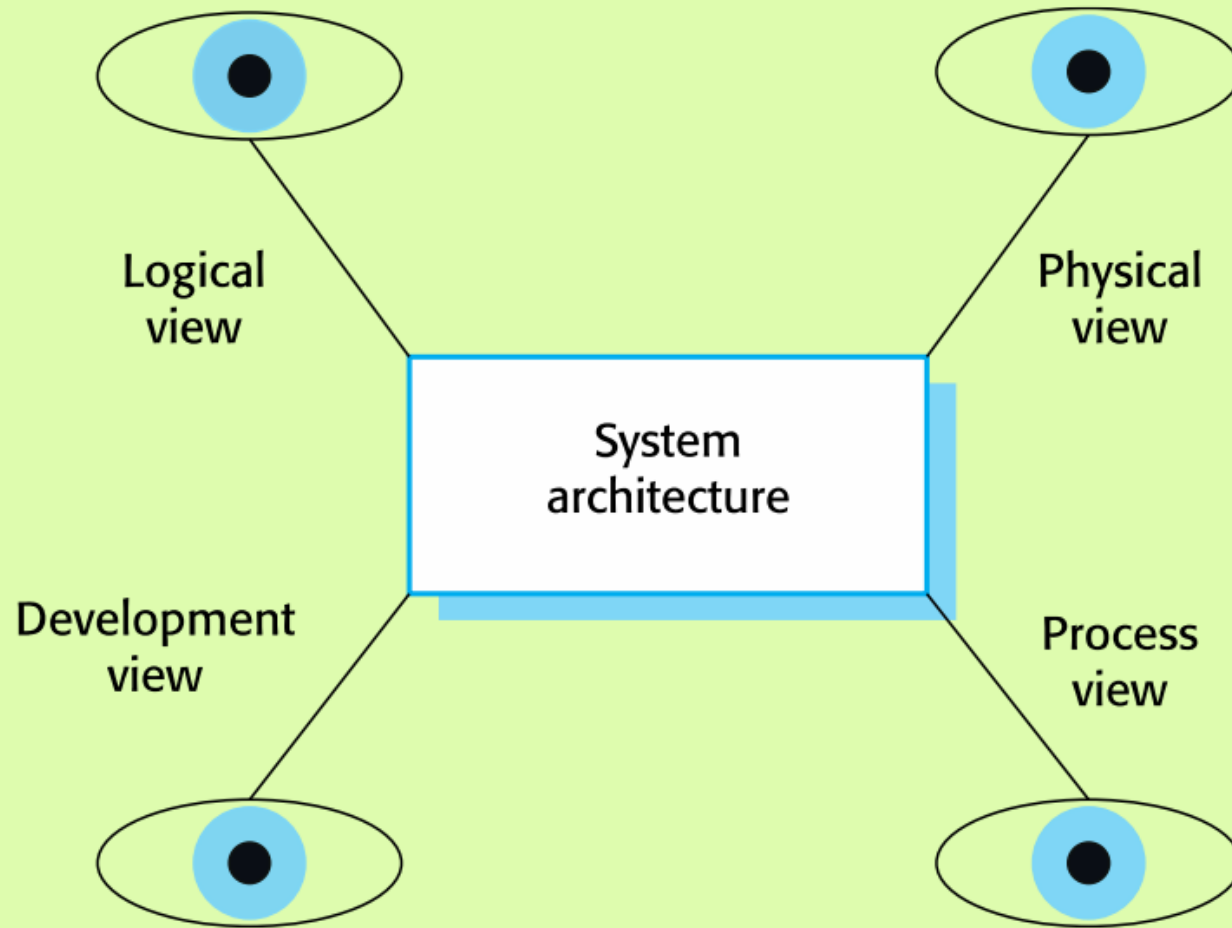
- **Performance**
 - Localize critical operations and minimize communications. Use large rather than fine-grain components.
- **Security**
 - Use a layered architecture with critical assets in the inner layers.
- **Safety**
 - Localize safety-critical features in a small number of sub-systems.
- **Availability**
 - Include redundant components and mechanisms for fault tolerance.
- **Maintainability**
 - Use fine-grain, replaceable components.



Architectural views

- What views or perspectives are useful when designing and documenting a system's architecture?
- What notations should be used for describing architectural models?
- Each architectural model only shows one view or perspective of the system.
 - It might show how a system is decomposed into modules, how the run-time processes interact or the different ways in which system components are distributed across a network. For both design and documentation, you usually need to present multiple views of the software architecture.

Architectural views



4 + 1 view model of software architecture

- A logical view, which shows the key abstractions in the system as objects or object classes.
- A process view, which shows how, at run-time, the system is composed of interacting processes.
- A development view, which shows how the software is decomposed for development.
- A physical view, which shows the system hardware and how software components are distributed across the processors in the system.
- Related using use cases or scenarios (+1)



Architectural patterns

- Patterns are a means of representing, sharing and reusing knowledge.
- An architectural pattern is a stylized description of good design practice, which has been tried and tested in different environments.
- Patterns should include information about when they are and when they are not useful.
- Patterns may be represented using tabular and graphical descriptions.

The Model-ViewController (MVC) pattern

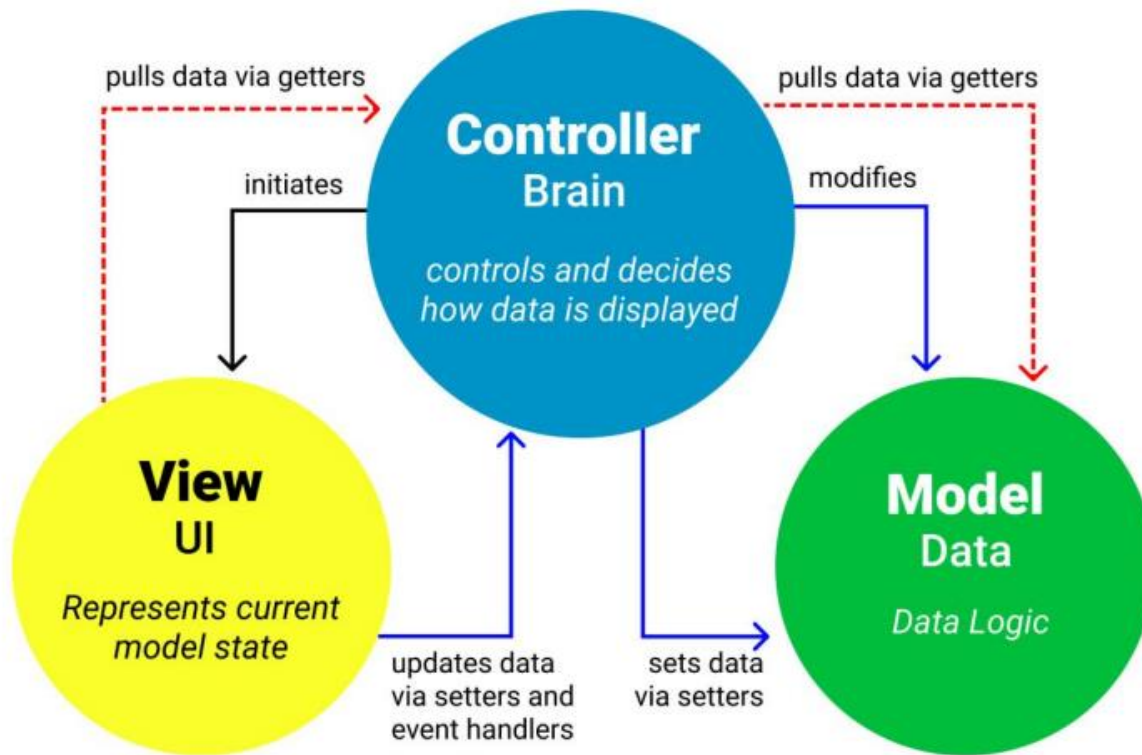
- Model-View-Controller pattern is a well-known pattern.
- This pattern is the basis of interaction management in many web-based systems and is supported by most language frameworks.
- The stylized pattern description includes the pattern name, a brief description, a graphical model, and an example of the type of system where the pattern is used.
- You should also include information about when the pattern should be used and its advantages and disadvantages.
- Graphical models of the architecture associated with the MVC pattern are shown in Fig. 6.5 and 6.6.
- These present the architecture from different views: Figure 6.5 is a conceptual view, and Figure 6.6 shows a runtime system architecture when this pattern is used for interaction management in a web-based system.

The Model-View-Controller (MVC) pattern

Name	MVC (Model-View-Controller)
Description	Separates presentation and interaction from the system data. The system is structured into three logical components that interact with each other. The Model component manages the system data and associated operations on that data. The View component defines and manages how the data is presented to the user. The Controller component manages user interaction (e.g., key presses, mouse clicks, etc.) and passes these interactions to the View and the Model. (See Figure 6.5).
Example	Figure 6.6 shows the architecture of a web-based application system organized using the MVC pattern.
When used	Used when there are multiple ways to view and interact with data. Also used when the future requirements for interaction and presentation of data are unknown.
Advantages	Allows the data to change independently of its representation and vice versa. Supports presentation of the same data in different ways with changes made in one representation shown in all of them.
Disadvantages	Can involve additional code and code complexity when the data model and interactions are simple.

The organization of the ModelView-Controller

MVC Architecture Pattern

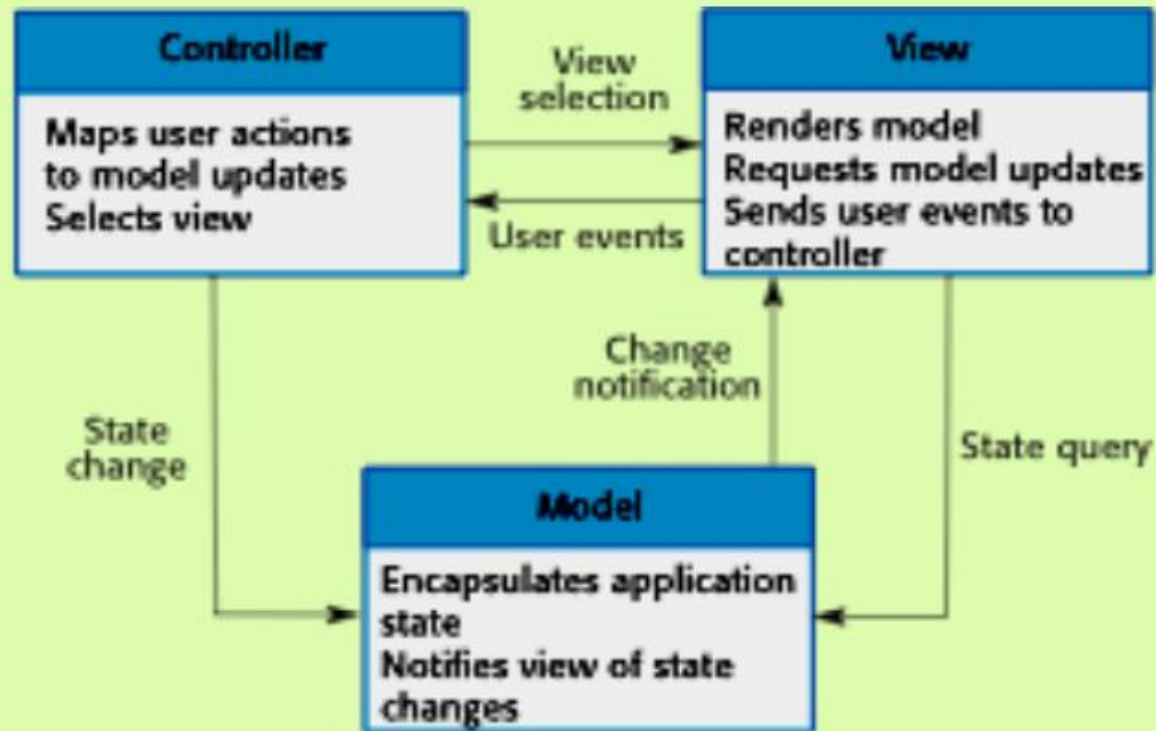


Model: The backend that contains all the data logic

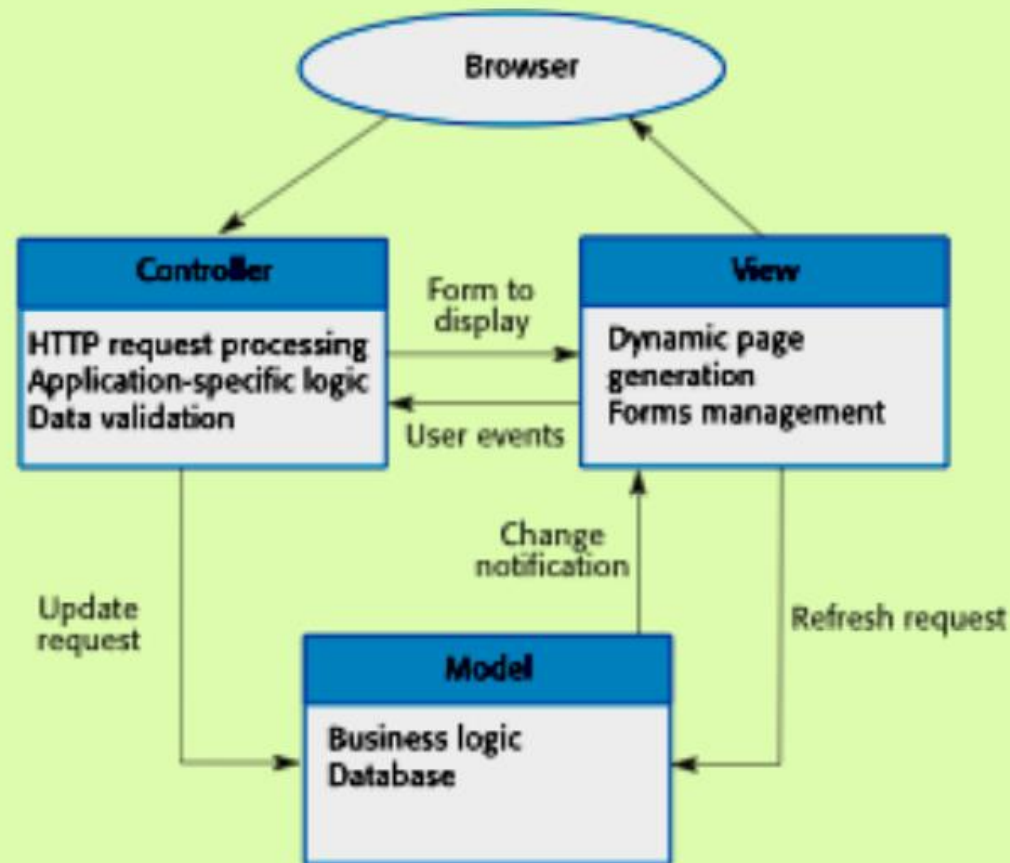
View: The frontend or graphical user interface (GUI)

Controller: The brains of the application that controls how data is displayed

The organization of the ModelView-Controller



Web application architecture using the MVC pattern



The Model-ViewController (MVC) pattern

- The MVC pattern, shown in previously (Figure 6.4) separates elements of a system, allowing them to change independently.
- For example, adding a new view or changing an existing view can be done without any changes to the underlying data in the model.



Why Should You Use MVC?

- The MVC pattern helps you break up the frontend and backend code into separate components.
- This way, it's much easier to manage and make changes to either side without them interfering with each other.



Layered architecture

- Used to model the interfacing of sub-systems.
- Organises the system into a set of layers (or abstract machines) each of which provide a set of services.
- Supports the incremental development of sub-systems in different layers. When a layer interface changes, only the adjacent layer is affected.
- However, often artificial to structure systems in this way.



Layered architecture

- This layered approach supports the incremental development of systems.
- As a layer is developed, some of the services provided by that layer may be made available to users.
- The architecture is also changeable and portable.
- If its interface is unchanged, a new layer with extended functionality can replace an existing layer without changing other parts of the system.



Layered architecture

- Furthermore, when layer interfaces change or new facilities are added to a layer, only the adjacent layer is affected.
- As layered systems localize machine dependencies, this makes it easier to provide multi-platform implementations of an application system.
- Only the machine-dependent layers need be reimplemented to take account of the facilities of a different operating system or database.

The Layered architecture pattern

Name	Layered architecture
Description	Organizes the system into layers with related functionality associated with each layer. A layer provides services to the layer above it so the lowest-level layers represent core services that are likely to be used throughout the system. See Figure 6.8.
Example	A layered model of a system for sharing copyright documents held in different libraries, as shown in Figure 6.9.
When used	Used when building new facilities on top of existing systems; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multi-level security.
Advantages	Allows replacement of entire layers so long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.
Disadvantages	In practice, providing a clean separation between layers is often difficult and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.

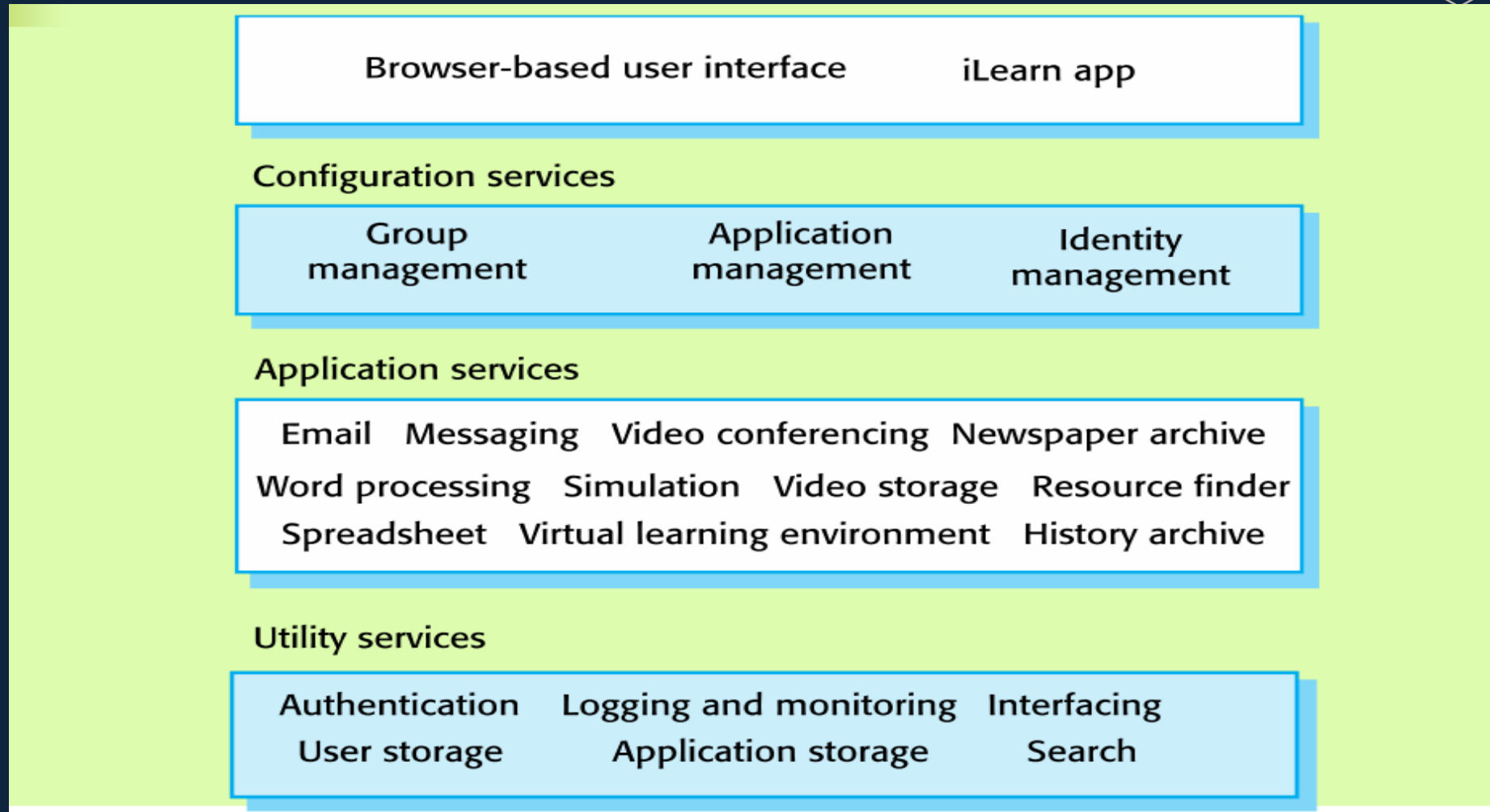
A generic layered architecture



A generic layered architecture

- Figure 6.8 is an example of a layered architecture with four layers.
- The lowest layer includes system support software—typically, database and operating system support.
- The next layer is the application layer, which includes the components concerned with the application functionality and utility components used by other application components.
- The third layer is concerned with user interface management and providing user authentication and authorization, with the top layer providing user interface facilities.
- Of course, the number of layers is arbitrary.

The architecture of the iLearn system





Repository architecture

- The Repository pattern (Figure 6.10), describes how a set of interacting components can share data.
- The majority of systems that use large amounts of data are organized around a shared database or repository.
- This model is therefore suited to applications in which data is generated by one component and used by another.
- Examples of this type of system include command and control systems, management information systems, Computer-Aided Design (CAD) systems, and interactive development environments for software.



Repository architecture

- Sub-systems must exchange data. This may be done in two ways:
 - Shared data is held in a central database or repository and may be accessed by all sub-systems;
 - Each sub-system maintains its own database and passes data explicitly to other sub-systems.
- When large amounts of data are to be shared, the repository model of sharing is most commonly used as this is an efficient data sharing mechanism.

The Repository pattern

Name	Repository
Description	All data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.
Example	Figure 6.9 is an example of an IDE where the components use a repository of system design information. Each software tool generates information which is then available for use by other tools.
When used	You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
Advantages	Components can be independent—they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.
Disadvantages	The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult

A repository architecture for an IDE

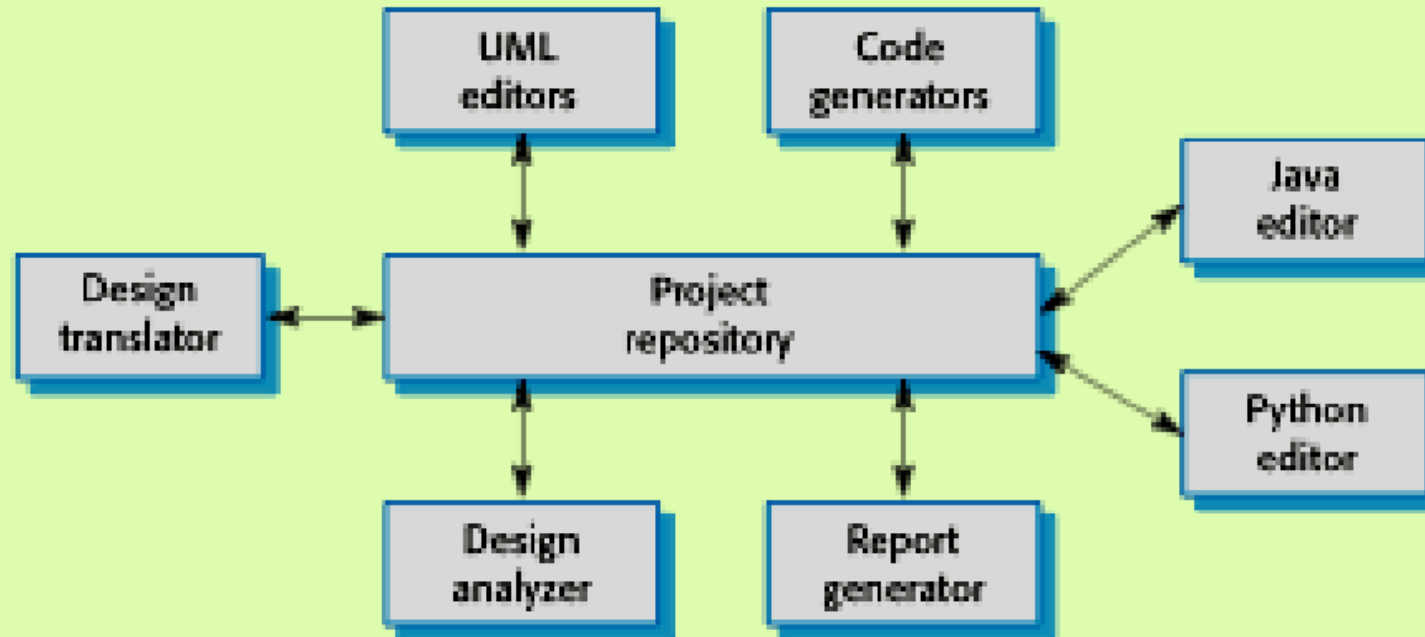


Figure 6.11 illustrates a situation in which a repository might be used. This diagram shows an IDE that includes different tools to support model-driven development. The repository in this case might be a version-controlled environment (as discussed in Chapter 25) that keeps track of changes to software and allows rollback to earlier versions.



Client-server architecture

- The Repository pattern is concerned with the static structure of a system and does not show its runtime organization.
- The Client–Server pattern (Figure 6.12), illustrates a commonly used runtime organization for distributed systems.
- A system that follows the Client–Server pattern is organized as a set of services and associated servers, and clients that access and use the services.



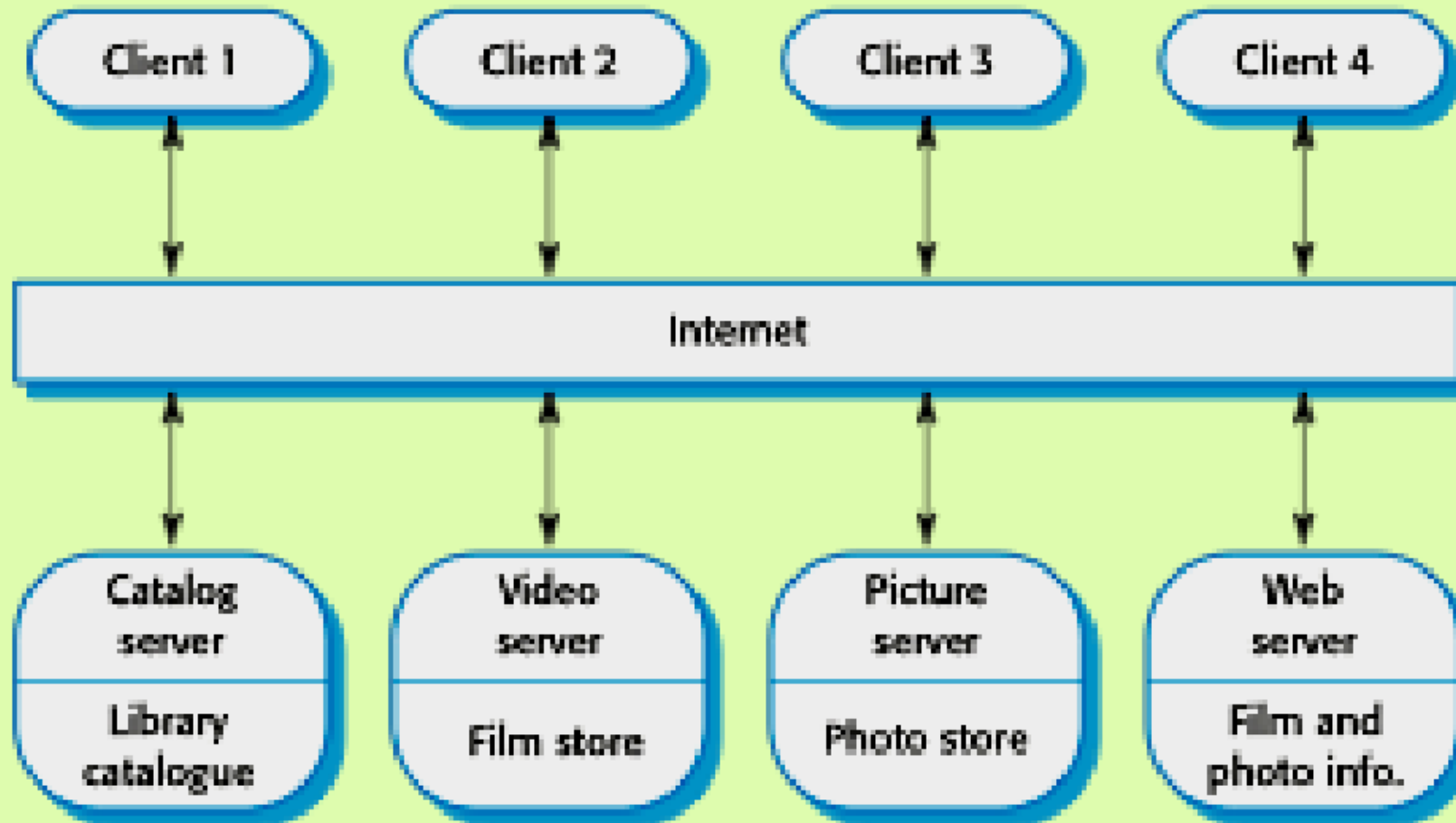
Client-server architecture

- Distributed system model which shows how data and processing is distributed across a range of components.
 - Can be implemented on a single computer.
- Set of stand-alone servers which provide specific services such as printing, data management, etc.
- Set of clients which call on these services.
- Network which allows clients to access servers.

The Client–server pattern

Name	Client-server
Description	In a client–server architecture, the functionality of the system is organized into services, with each service delivered from a separate server. Clients are users of these services and access servers to make use of them.
Example	Figure 6.13 is an example of a film and video/DVD library organized as a client–server system.
When used	Used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.
Advantages	The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients and does not need to be implemented by all services.
Disadvantages	Each service is a single point of failure so susceptible to denial of service attacks or server failure. Performance may be unpredictable because it depends on the network as well as the system. May be management problems if servers are owned by different organizations.

A client-server architecture for a film library





Pipe and filter architecture

- The Pipe and Filter pattern (Figure 6.14) is a model of the runtime organization of a system where functional transformations process their inputs and produce outputs.
- Data flows from one to another and is transformed as it moves through the sequence.
- Each processing step is implemented as a transform. Input data flows through these transforms until converted to output.
- The transformations may execute sequentially or in parallel.
- The data can be processed by each transform item by item or in a single batch.



Pipe and filter architecture

- The name “pipe and filter” comes from the original Unix system where it was possible to link processes using “pipes.”
- These passed a text stream from one process to another.
- Systems that conform to this model can be implemented by combining Unix commands, using pipes and the control facilities of the Unix shell.
- The term filter is used because a transformation “filters out” the data it can process from its input data stream.



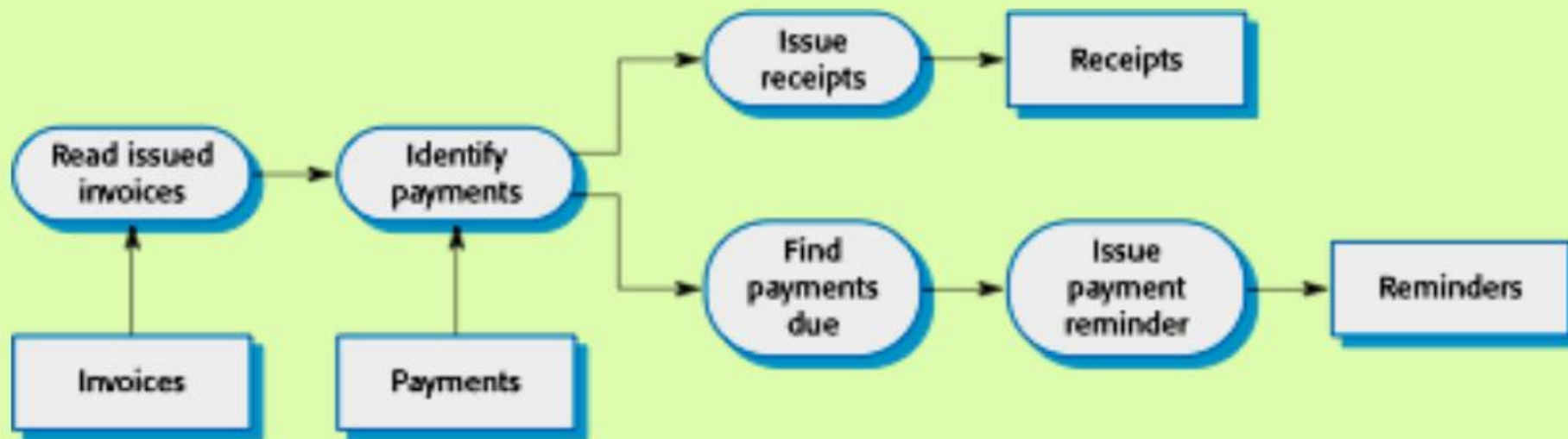
Pipe and filter architecture

- Functional transformations process their inputs to produce outputs.
- May be referred to as a pipe and filter model (as in UNIX shell).
- Variants of this approach are very common. When transformations are sequential, this is a batch sequential model which is extensively used in data processing systems.
- Not really suitable for interactive systems.

The pipe and filter pattern

Name	Pipe and filter
Description	The processing of the data in a system is organized so that each processing component (filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.
Example	Figure 6.15 is an example of a pipe and filter system used for processing invoices.
When used	Commonly used in data processing applications (both batch- and transaction-based) where inputs are processed in separate stages to generate related outputs.
Advantages	Easy to understand and supports transformation reuse. Workflow style matches the structure of many business processes. Evolution by adding transformations is straightforward. Can be implemented as either a sequential or concurrent system.
Disadvantages	The format for data transfer has to be agreed upon between communicating transformations. Each transformation must parse its input and unparse its output to the agreed form. This increases system overhead and may mean that it is impossible to reuse functional transformations that use incompatible data structures.

An example of the pipe and filter architecture





Application architectures

- Application systems are designed to meet an organizational need.
- As businesses have much in common, their application systems also tend to have a common architecture that reflects the application requirements.
- A generic application architecture is an architecture for a type of software system that may be configured and adapted to create a system that meets specific requirements.



Use of application architectures

- As a starting point for architectural design process
- As a design checklist
- As a way of organizing the work of the development team
- As a means of assessing components for reuse
- As a vocabulary for talking about applications

Use of application architectures

- *As a starting point for architectural design process* If you are unfamiliar with the types of application that you are developing, you can based your initial design on a generic application architecture. You then specialize this for the specific system that is being developed.
- *As a design checklist* If you have developed an architectural design for an application system, you can compare this with the generic application architecture. You can check that your design is consistent with generic architecture.
- *As a way of organizing the work of the development team* The application architecture identify stable structural features of the system architecture, and in many cases, it is possible to develop these in parallel. You can assign work to group members to implement different components with the architecture.

Use of application architectures(Cont.)

- *As a means of assessing components for reuse* If you have components you might be able to reuse, you can compare these with the generic structures to see whether there are comparable components in the application architecture.
- *As a vocabulary for talking about applications* If you are discussing a specific application or trying to compare applications, then you can use the concepts identified in the generic architecture to talk about these applications.

Examples of application types

- *Data processing applications*
 - Data driven applications that process data in batches without explicit user intervention during the processing.
- *Transaction processing applications*
 - Data-centred applications that process user requests and update information in a system database.
- *Event processing systems*
 - Applications where system actions depend on interpreting events from the system's environment.
- *Language processing systems*
 - Applications where the users' intentions are specified in a formal language that is processed and interpreted by the system.



Application type examples

- Focus here is on transaction processing and language processing systems.
- Transaction processing systems
 - E-commerce systems;
 - Reservation systems.
- Language processing systems
 - Compilers;
 - Command interpreters.

Transaction processing systems

- Transaction processing systems are designed to process user requests for information from a database, or requests to update a database.
- Technically, a database transaction is part of a sequence of operations and is treated as a single unit (an atomic unit).
- All of the operations in a transaction have to be completed before the database changes are made permanent.
- This ensures that failure of operations within a transaction does not lead to inconsistencies in the database.

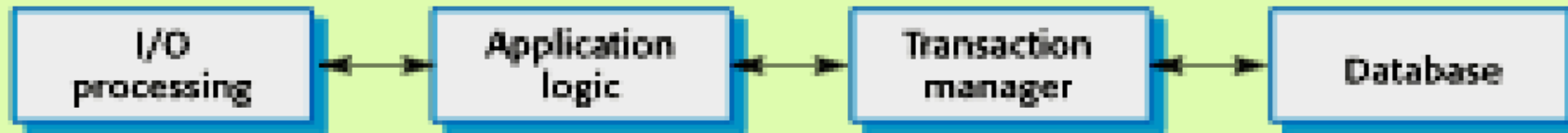
Transaction processing systems

- Process user requests for information from a database or requests to update the database.
- From a user perspective a transaction is:
 - Any coherent sequence of operations that satisfies a goal;
 - For example - find the times of flights from London to Paris.
- Users make asynchronous requests for service which are then processed by a transaction manager.

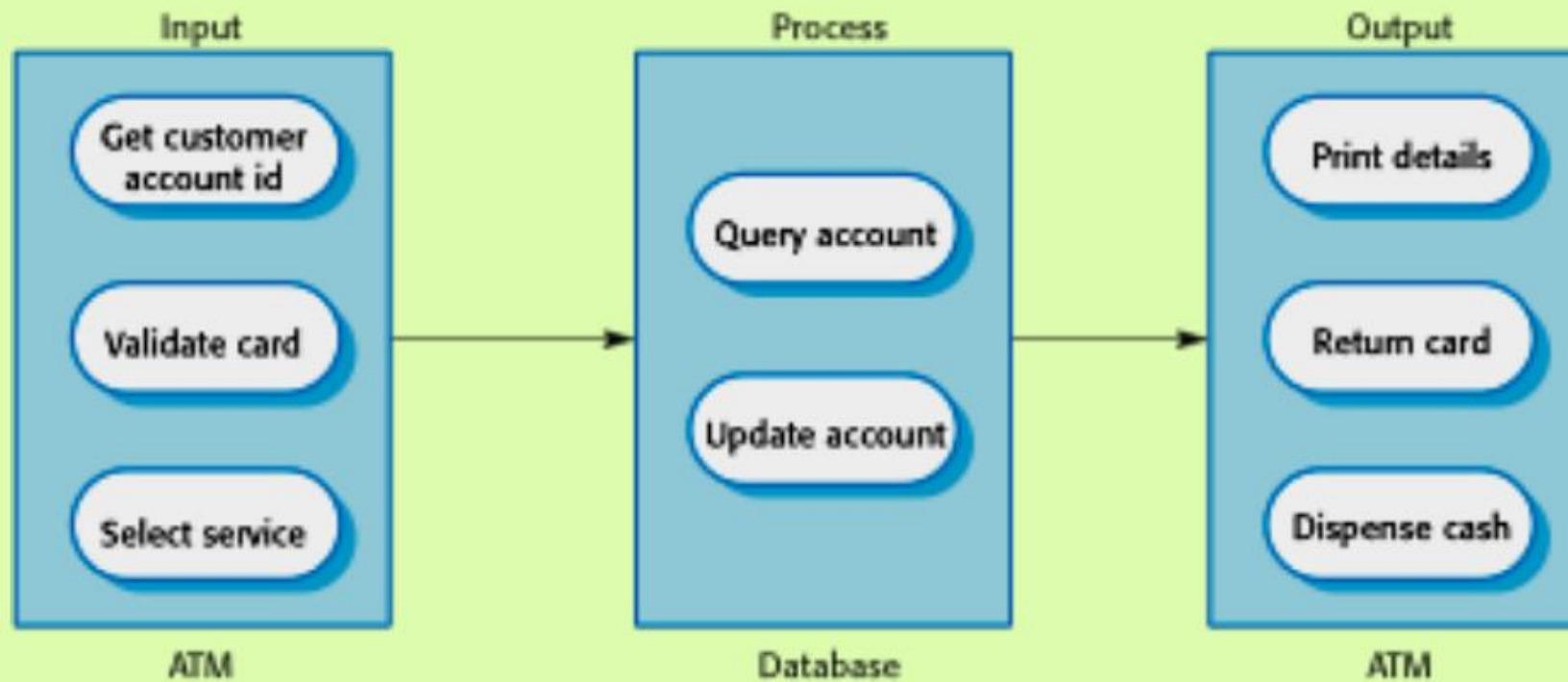
Transaction processing systems

- Transaction processing systems are usually interactive systems in which users make asynchronous requests for service.
- Figure 6.16 illustrates the conceptual architectural structure of transaction processing applications.
 - First, a user makes a request to the system through an I/O processing component.
 - The request is processed by some application-specific logic.
 - A transaction is created and passed to a transaction manager, which is usually embedded in the database management system.
 - After the transaction manager has ensured that the transaction is properly completed, it signals to the application that processing has finished.
- Transaction processing systems may be organized as a “pipe and filter” architecture, with system components responsible for input, processing, and output.

The structure of transaction processing applications



The software architecture of an ATM system



Information systems architecture

- Information systems have a generic architecture that can be organized as a layered architecture.
- These are transaction-based systems as interaction with these systems generally involves database transactions.
- Information systems are almost always web based systems, where the user interface is implemented in a web browser.
- Layers include:
 - The user interface
 - User communications
 - Information retrieval
 - System database

Layered information system architecture

- The system is modeled using a layered approach where the top layer supports the user interface and the bottom layer is the system database.
- The user communications layer handles all input and output from the user interface.
- The information retrieval layer includes application-specific logic for accessing and updating the database.
- The layers in this model can map directly onto servers in a distributed Internet-based system.



User Interface



User Communication

Authentication and authorization



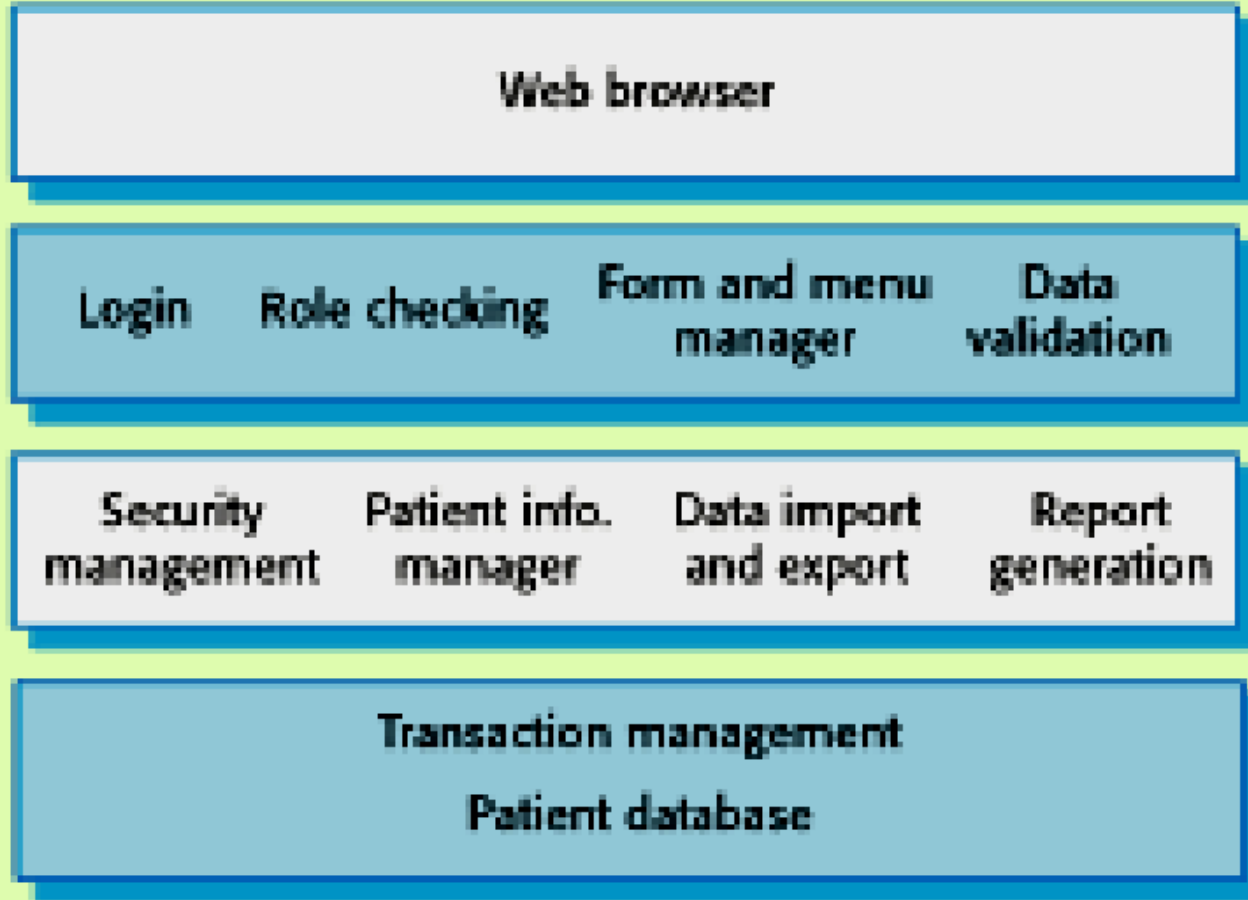
Information retrieval and modification



Transaction management Databases



The architecture of the MHC-PMS



- Top layer is a browser-based user interface
- The second layer is about the user interface functionality
- Third layer implements the functionality of the system
- The fourth layer is commercial database management system

Web-based information systems

- Information and resource management systems are now usually web-based systems where the user interfaces are implemented using a web browser.
- For example, e-commerce systems are Internet-based resource management systems that accept electronic orders for goods or services and then arrange delivery of these goods or services to the customer.
- In an e-commerce system, the application-specific layer includes additional functionality supporting a 'shopping cart' in which users can place a number of items in separate transactions, then pay for them all together in a single transaction.



Server implementation

- These systems are often implemented as multi-tier client server/architectures .
 - The web server is responsible for all user communications, with the user interface implemented using a web browser;
 - The application server is responsible for implementing application-specific logic as well as information storage and retrieval requests;
 - The database server moves information to and from the database and handles transaction management.

A decorative header consisting of a row of hexagons in various shades of blue, white, and orange, with some hexagons having orange outlines.

Language processing systems

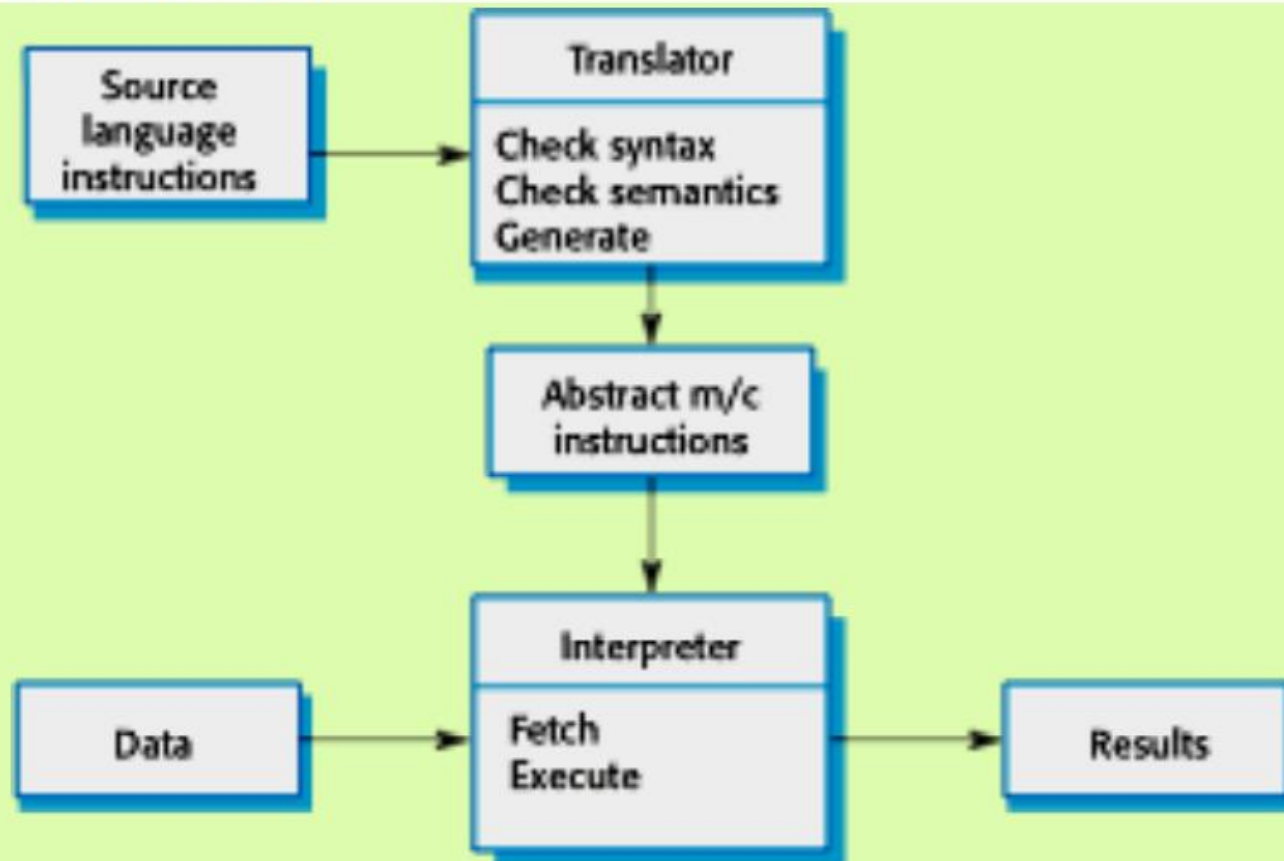
- Language processing systems translate one language into an alternative representation of that language and, for programming languages, may also execute the resulting code.
- Compilers translate a programming language into machine code.
- Other language processing systems may translate an XML data description into commands to query a database or to an alternative XML representation.
- Natural language processing systems may translate one natural language to another, for example, Urdu to English.

Language processing systems

- Accept a natural or artificial language as input and generate some other representation of that language.
- May include an interpreter to act on the instructions in the language that is being processed.
- Used in situations where the easiest way to solve a problem is to describe an algorithm or describe the system data
 - Meta-case tools process tool descriptions, method rules, etc and generate tools.

MDA transformations

A possible architecture for a language processing system for a programming language is illustrated as



Language processing systems

- The source language instructions define the program to be executed, and a translator converts these into instructions for an abstract machine.
- These instructions are then interpreted by another component that fetches the instructions for execution and executes them using (if necessary) data from the environment.
- The output of the process is the result of interpreting the instructions on the input data.
- For many compilers, the interpreter is the system hardware that processes machine instructions, and the abstract machine is a real processor.
- However, for dynamically typed languages, such as Ruby or Python, the interpreter is a software component.



Compiler components

- A **lexical analyzer**, which takes input language tokens and converts them to an internal form.
- A **symbol table**, which holds information about the names of entities (variables, class names, object names, etc.) used in the text that is being translated.
- A **syntax analyzer**, which checks the syntax of the language being translated.



Compiler components (Cont.)

- A **syntax tree**, which is an internal structure representing the program being compiled.
- A **semantic analyzer** that uses information from the syntax tree and the symbol table to check the semantic correctness of the input language text.
- A **code generator** that ‘walks’ the syntax tree and generates abstract machine code.

A repository architecture for a language processing system

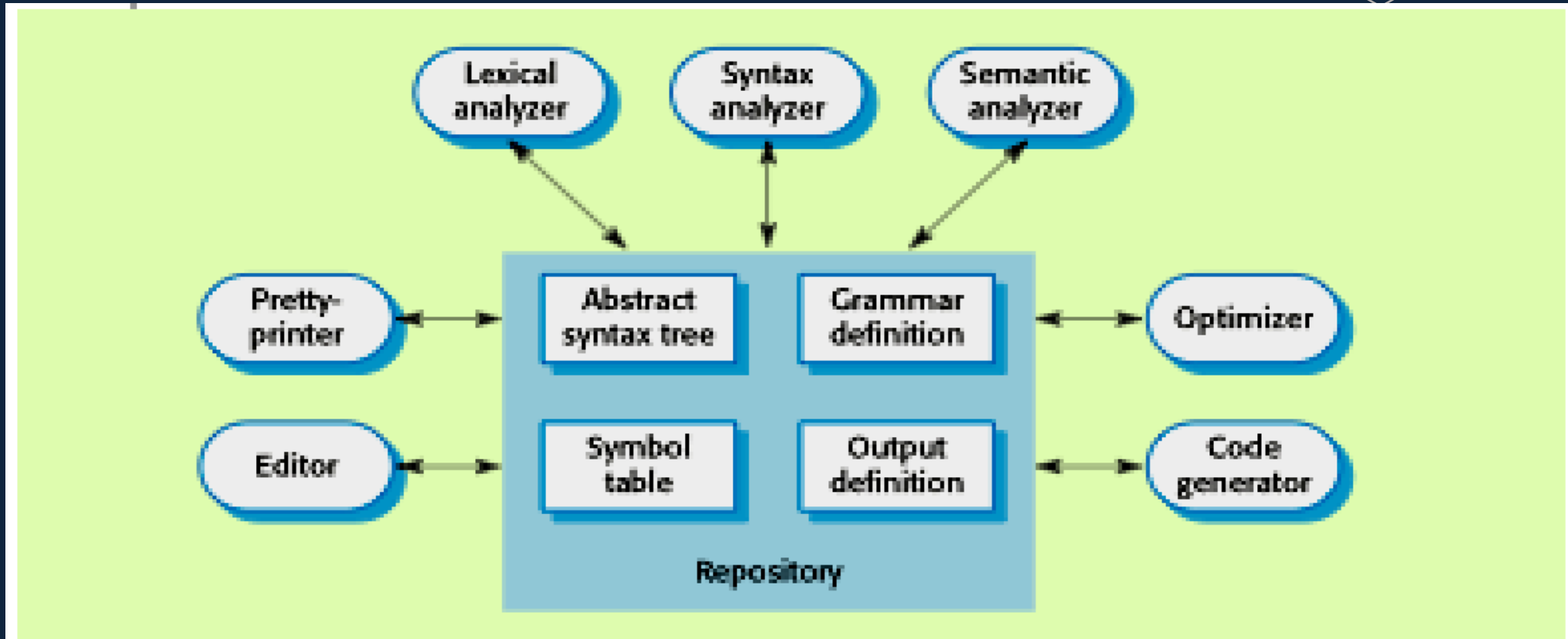


Figure 6.21 illustrates how a language processing system can be part of an integrated set of programming support tools.

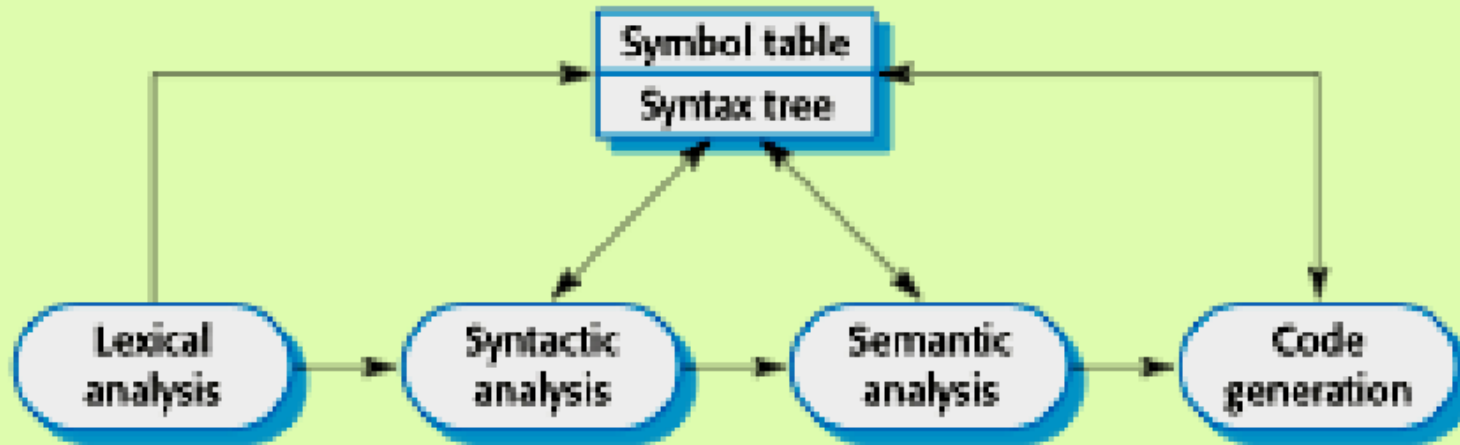
Model driven architecture(Cont.)



- In this example, the symbol table and syntax tree act as a central information repository.
- Tools or tool fragments communicate through it.
- Other information that is sometimes embedded in tools, such as the grammar definition and the definition of the output format for the program, have been taken out of the tools and put into the repository.
- Therefore, a syntax-directed editor can check that the syntax of a program is correct as it is being typed.
- A program formatter can create listings of the program that highlight different syntactic elements and are therefore easier to read and understand.

A pipe and filter compiler architecture

- Pipe and filter model of language compilation is effective in batch environments where programs are compiled and executed without user interactions.





Key points

- A software architecture is a description of how a software system is organized.
- Architectural design decisions include decisions on the type of application, the distribution of the system, the architectural styles to be used.
- The design process for identifying the sub-systems making up a system and the framework for subsystem control and communication is architectural design.



Key points

- Architectures may be documented from several different perspectives or views such as a conceptual view, a logical view, a process view, and a development view.
- Architectural patterns are a means of reusing knowledge about generic system architectures. They describe the architecture, explain when it may be used and describe its advantages and disadvantages.



Key points

- Models of application systems architectures help us understand and compare applications, validate application system designs and assess large-scale components for reuse.
- Transaction processing systems are interactive systems that allow information in a database to be remotely accessed and modified by a number of users.
- Language processing systems are used to translate texts from one language into another and to carry out the instructions specified in the input language. They include a translator and an abstract machine that executes the generated language.

A decorative pattern of hexagons in various shades of blue, orange, and white, arranged in a honeycomb-like structure on the left side of the slide.

Thank you

Prof. Dr. Fahim Akhtar Rajput